Style Transfer with Deep Neural Networks Report

Overview: In this project, I explored neural style transfer using Convolutional Neural Networks. This approach uses a pre-trained deep neural network to extract and recombine features from two different images.

The objective of this project is to understand how CNNs can separate content and style representations of images and combine them to generate a new stylized output image.

Implementation: The images used in this project were chosen by me:

```
In [5]: content = load_image(
            "https://i.redd.it/with-the-design-reveal-of-evanescia-and-sparxie-it-feels-v0-1cu4wje40ndg1.jpg?width=1920&format=pjpg&auto=
        ).to(device)

        style = load_image(
            "https://static.beebom.com/wp-content/uploads/2026/01/Yao-Guang-Honkai-Star-Rail-Planarcadia.jpg?w=1024&quality=75",
            shape=content.shape[-2:]
        ).to(device)
```

```
Out[7]: <matplotlib.image.AxesImage at 0x1fa6e2b1c70>
```

- A pre-trained VGG19 model was used for feature extraction
- Feature extraction layers of the network were utilized
- The model parameters were frozen

The input images were preprocessed as follows:

- Resized to 400 pixels
- Normalized using ImageNet mean and standard deviation

Feature extraction was performed at specific layers of the network:

- Content representation: conv4_2
- Style representation: conv1_1, conv2_1, conv3_1, conv4_1, conv5_1

```
if layers is None:
    layers = {'0': 'conv1_1',
              '5': 'conv2_1',
              '10': 'conv3_1',
              '19': 'conv4_1',
              '21': 'conv4_2',
              '28': 'conv5_1'
             }
```

The Gram Matrix was used:

- Captures the correlations between feature maps and removes spatial information

The style transfer process was performed:

- The target image was initialized as a copy of the content image
- The A dam optimizer was used to update the image iteratively
- The model was run for approximately 2000 iterations

```
plt.imshow(im_convert(target))
plt.show()
```

```
Total loss:  74161152.0
```

Display the Target Image

Three types of losses were computed, and the weights used are:

- Content Loss: Measures the difference between target and content features
- Style Loss: Measures the difference between the Gram matrices of the target and style features
- Total Loss: Combination of content and style losses
- Content weight (α) = 1
- Style weight (β) = 1e6

```
for ii in range(1, steps+1):

    ## TODO: get the features from your target image
    ## Then calculate the content loss
    target_features = get_features(target, vgg)

    # content loss
    content_loss = torch.mean(
        (target_features['conv4_2'] - content_features['conv4_2']) ** 2
    )

    # style loss
    style_loss = 0

    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        # target gram
        target_gram = gram_matrix(target_feature)

        # style gram
        style_gram = style_grams[layer]

        # layer style loss
        layer_style_loss = style_weights[layer] * torch.mean(
            (target_gram - style_gram) ** 2
        )

        style_loss += layer_style_loss / (d * h * w)

    ## TODO:  calculate the *total* loss
    total_loss = content_weight * content_loss + style_weight * style_loss
```

Results: The results showed that initially, the target image appeared similar to the content image. During training, style patterns gradually appeared. Colors and textures from the style image were applied to the target image.

Conclusion: The project was concluded by preserving the overall structure of the content image and incorporating style features from the image. The time taken for the process was approximately 15 minutes, as the number of iterations took a while to process.

Findings: From this experiment, I found that:

- Early layers of the network detect low-level features such as edges and colors
- Deeper layers capture high-level features such as object shapes
- The Gram matrix focuses on texture patterns
- Higher style weight results in stronger stylization but may distort content
- Lower style weight preserves structure but reduces the strength of the style

- Loss values decreased over iterations

What Helps: Some presumptions I believe that can be used to improve the project results are using multiple layers for style extraction to capture both coarse and fine textures. Freezing the model to improve efficiency and reduce computation could also show improvements. I also believe that properly balancing content and style weights and running more iterations can result in better visual quality.

Conclusion: Unlike traditional image processing methods that rely on manually designed filters, CNNs automatically learn meaningful features from data. Overall, this project shows how deep learning can be applied creatively beyond standard classification tasks.

References

- https://pytorch.org/docs/stable/index.html
- https://docs.opencv.org/
- https://matplotlib.org/stable/contents.html